CALIFORNIA STATE UNIVERSITY
LONG BEACH

CECS 341: Computer Architecture & Organization

Hossein Sayadi, PhD

Department of Computer Engineering & Computer Science
California State University, Long Beach

Fall 2025

Processor Performance

Lecture Outline

- Performance Basics
  - Time and Space metrics
  - Performance as related to execution time (ET)
- How to calculate clock cycles and execution time
- How to improve the processor performance
  - Decrease instruction count, reduce CPI_{AVG}, increase clock frequency.

Performance Basics

Time and Space metrics

Performance as related to execution time (ET)

Performance

- Performance is relative evaluation metric.
  - It is dependent on what is being important to the computer system's endpoint user.
- There are different evaluation metrics used to characterize and compare the processors' performance.
  - Time-related metrics
  - Space-related metrics
- Each evaluation metric is based on a different measurement of interest.

Processor Performance

- Two important performance metrics are used for program execution:

1. Time metric
   - Amount of time required to complete execution of a program.
   - aka. Execution time (CPU time)
   - Linear function of the number of (machine) instructions executed.

2. Space metric
   - Amount of memory required by a program for instructions and data.
   - aka. Memory footprint.
   - Not necessarily correlated to the program execution time.

Time Metrics

- Instruction Latency
  - Time required to complete a single instruction.
    - Includes *fetch*, *decode*, and *execution* of instruction.
  - Usually measured in *seconds*.
- Execution Time (ET)
  - Time required to complete a program or a code segment.
    - Includes *fetch*, *decode*, and *execution* of all instructions.
  - Usually measured in *seconds* or *seconds per instruction*.
- Throughput
  - Number of instructions completed per unit time.
  - Usually measured in *instructions per second*.

Processor Performance (Time metric)

- The performance of a processor is more than just the speed (or frequency) of the processor clock.
- Must take into consideration the effects of the instruction set.
  - Simple or complex instructions.
  - How many micro-operations are needed to process each instruction?
- Must take into consideration the effects of the datapath
  - Can micro-operations be completed in parallel?
  - How many clock cycles are required to process each instruction?
  - What is the critical path in the processor datapath, and what is the resulting clock period?

Execution Time

- Execution time is the best measure of processor performance.
  - Characterize processor performance.
  - Compare performance of different processors.
  - Compare performance before and after a modification to a processor.

- Minimize program ET to maximize processor performance:

$$\text{Performance} = \frac{1}{\text{Execution Time}}$$

- When comparing performance of two processors (X and Y):

$\text{Performance}_X > \text{Performance}_Y$ if $ET_X < ET_Y$

Comparing Processors

- N_{XY} is defined to be the ratio of the performance of processor X to the performance of processor Y:

$$N_{XY} = \frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution Time}_Y}{\text{Execution Time}_X}$$

- If $N_{XY} > 1$
  - Processor X is faster than Processor Y by a factor of N_{XY}.
- If $N_{XY} < 1$
  - Processor Y is faster than Processor X by a factor of $1/N_{XY}$.

N_{XY} = speedup of processor X over processor Y

Evaluating Improvements

- A modification is made to the processor.
- The resulting *speedup*, or improvement, can be described by

$$N_{IMPROVE} = \frac{Performance_{AFTER}}{Performance_{BEFORE}} = \frac{Execution\ Time_{BEFORE}}{Execution\ Time_{AFTER}}$$

- If $N_{IMPROVE} > 1$
  - Processor performance is ***increased*** by a factor of $N_{IMPROVE}$, because of the modification made to the processor.
- If $N_{IMPROVE} < 1$
  - Processor performance is ***decreased*** by a factor of $N_{IMPROVE}$.

$N_{IMPROVE}$ = speedup of processor after improvement

Clock Cycles and Execution Time

Terminology

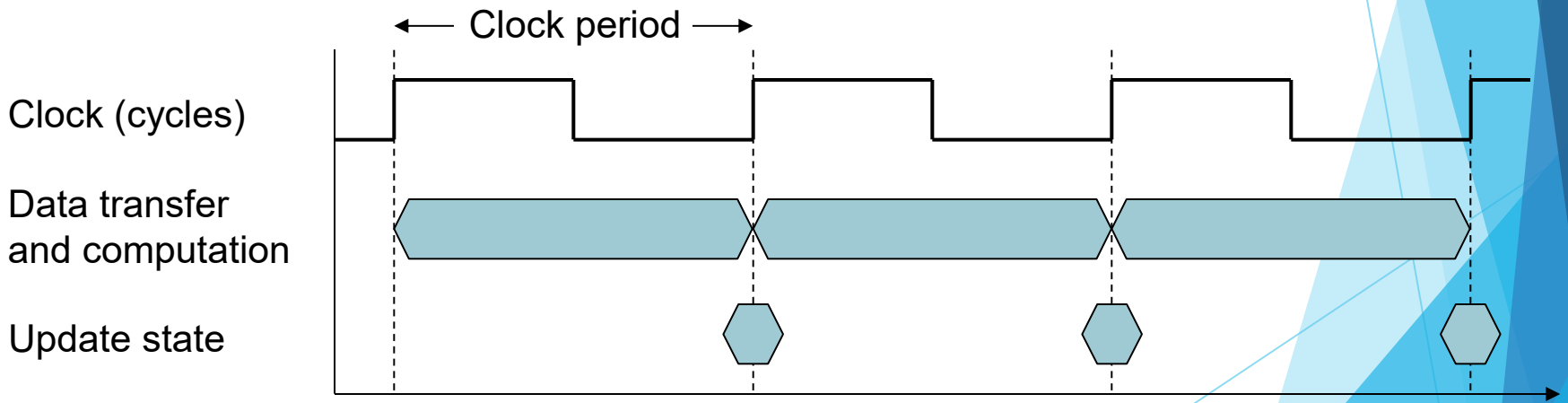
Average clock cycles per Instruction (CPI_{AVG})

Total clock cycles (CC_{TOT})

Execution Time (ET)

Processor Clock Cycles

- Processor operation is governed by a constant-rate clock.
  - Clock period is determined by the critical path in the processor.
- Instruction cycle is controlled by the clock.
  - Each step (or micro-operation) requires one clock cycle.
  - Instruction processing is described using Register Transfer Language (RTL).

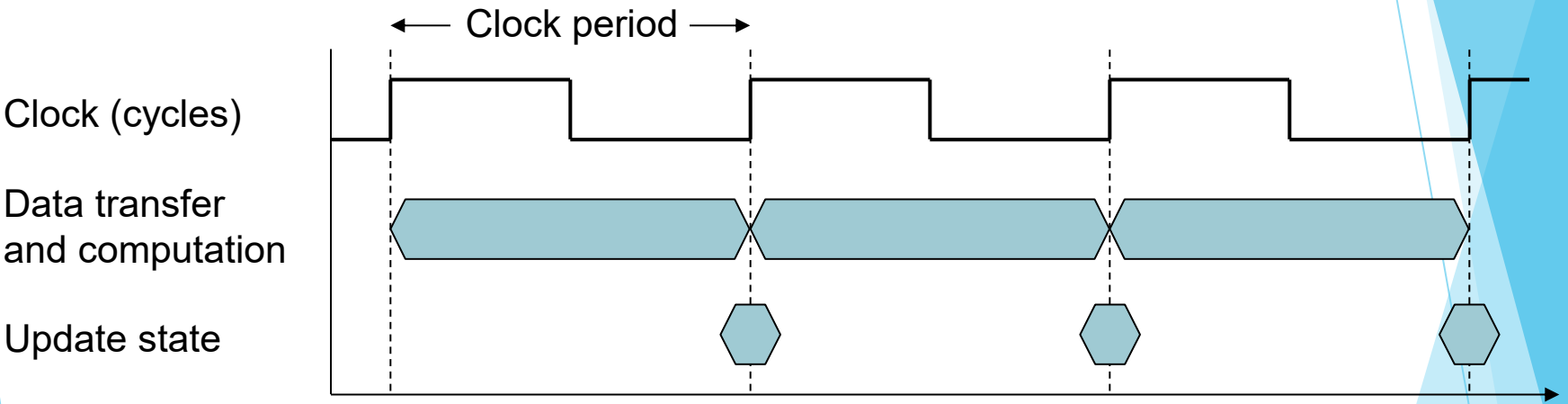


T_{CLK} = clock period (clock cycle time)

F_{CLK} = clock frequency (clock rate)

Processor Clock Cycles

- Processor operation is governed by a constant-rate clock.
  - Clock period is determined by the critical path in the processor.



- Clock period: duration of a clock cycle
  - e.g., 250ps = 0.25ns = 250×10^{-12}s
- Clock frequency (rate): cycles per second
  - e.g., 4.0GHz = 4000MHz = 4.0×10^{9}Hz

Terminology

- Instruction Count (IC)
  - Number of (machine) instructions in a specified program.
- Clock cycles per Instruction (CPI)
  - Number of clock cycles required to complete each instruction.
  - Different instructions may require a different number of clock cycles.
  - Integer value.
- CPI for class k (CPI_k)
  - Number of clock cycles required to complete an instruction of class *k*.
- Instruction Frequency (F_k)
  - Frequency of an instruction of class *k* in the program being executed.

Terminology

- Average clock cycles per Instruction (CPI_{AVG})
  - Average number of clock cycles required to execute each instruction in the specified program.
  - CPI_{AVG} is **program specific**.
  - Can be a real number with a fractional part.

- Total clock cycles (CC_{TOT})
  - Total number of clock cycles required to complete the program.
  - CC_{TOT} is **program specific**.
  - It varies from one program to another.

Calculating Average CPI (CPI_{AVG})

- Average CPI can be calculated using CPI and IC:

$$CPI_{AVG} = \left(\sum_{i=1}^{IC} CPI_i\right) / IC$$

- It can also be calculated as a weighted average:

$$CPI_{AVG} = (F_A * CPI_A + F_B * CPI_B + \ldots)$$

$$CPI = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^{n} \left(CPI_i \times \underbrace{\frac{\text{Instruction Count}_i}{\text{Instruction Count}}}_{\text{Relative frequency}} \right)$$

Instruction Class	Frequency in Program	CPI
A	F_A	CPI_A
B	F_B	CPI_B
C	F_C	CPI_C

CPI Example

- Alternative compiled code sequences using instructions in classes A, B, C

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- Sequence 1: IC = 5
  - Clock Cycles
    = 2×1 + 1×2 + 2×3
    = 10
  - Avg. CPI = 10/5 = 2.0
- Sequence 2: IC = 6
  - Clock Cycles
    = 4×1 + 1×2 + 1×3
    = 9
  - Avg. CPI = 9/6 = 1.5

Calculating Total Clock Cycles (CPI_{TOT})

- Total clock cycles can be calculated using CPI and IC:

$$CC_{TOT} = \sum_{i=1}^{IC} CPI_i$$

- It can also be calculated using CPI_{AVG} and IC:

$$CC_{TOT} = CPI_{AVG} * IC$$

Calculating Execution Time (ET)

- Execution time is a function of
  - The total number of clock cycles required to execute the program.
  - The speed (or clock period) of the processor.

- Execution time (ET) can be calculated as follows:

$$ET = CC_{TOT} * T_{CLK}$$

- Clock Period (T_{CLK})
  - Determined by the critical path in the datapath.
  - Technology dependent.
  - Related to the speed of the processor.

Improving Processor Performance

Decrease instruction count.

Reduce average clock cycle per instruction.

Increase clock frequency (decrease clock period).

Improving Processor Performance

$$ET = \underset{(1)}{IC} * \underset{(2)}{CPI_{AVG}} * \underset{(3)}{T_{CLK}}$$

- Processor performance can be improved by

1) Decreasing the instruction count.
2) Reducing the average clock cycles per instruction.
3) Increasing the clock frequency (decreasing the clock period).

Decreasing Instruction Count (IC)

- The instruction count (IC) for a ML program is a function of:
  - Number of instructions in the HLL program.
  - Instruction set architecture (ISA) of the processor.
  - (the efficiency of the) Compiler.

- Replace simple instructions with more complex ones.
  - RISC vs. CISC debate.
  - Reduces the number of (machine) instructions executed in the program.
  - May have a negative impact on CPI_{AVG} and clock period (T_{CLK}).

- Compiler Optimization
  - Plays a critical role in the execution time of long-running programs.

Reducing Average CPI (CPI_{AVG})

- The average CPI for a ML program is a function of:
  - The processor hardware (i.e. the datapath and control unit).
  - The instruction mix in the program.

- One-bus organization limits hardware concurrency.
- Increase concurrency by using multiple buses.
  - Must consider multiple delay paths when determining clock period.
  - May have a negative impact on clock period (T_{CLK}).

- Optimize the microarchitecture of the processor.
  - Minimize number of steps needed to implement each machine instruction.
  - Optimize the control unit.

Increasing Clock Frequency

- The processor clock frequency (i.e. clock speed) is determined by the critical path of the processor datapath.
- Rearrange datapath components to reduce delay.
  - Physical layout of processor.
- Reduce number of datapath actions taken in one clock cycle.
  - Impacts the CPI required to execute each instruction in the ISA.
- Reduce the feature size.
  - New fabrication processes and device technologies.

Improving Processor Performance

$$ET = \underset{(1)}{IC} * \underset{(2)}{CPI_{AVG}} * \underset{(3)}{T_{CLK}}$$

- All three factors of program execution time are interrelated.
- They must be optimized simultaneously, not in isolation.
- Must consider trade-offs.

CPI Example

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same ISA
- Which is faster, and by how much?

CPI Example

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same ISA
- Which is faster, and by how much?

$$\text{CPU Time}_A = \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A$$
$$= I \times 2.0 \times 250\text{ps} = I \times 500\text{ps}$$

A is faster…

$$\text{CPU Time}_B = \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B$$
$$= I \times 1.2 \times 500\text{ps} = I \times 600\text{ps}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{I \times 600\text{ps}}{I \times 500\text{ps}} = 1.2$$

…by this much